

CS 162

Computer Architecture

Winter 2026

Due: 11:59pm Jan 21st, 2026

Points: 100

1. (25 pts) The speedup is determined by the percentage of the application that can be parallelized and the additional cost of communication between parallel processes. Amdahl's law takes into account the former but not the latter.

a) What is the speedup with 8 processors if 64% of the application is parallelizable and for every processor used, the communication overhead is 1% of the original execution time?

b) Generalize the speedup formula from part (a) to an arbitrary number of processors, N , and determine the number of processors that will result in the highest speedup in the application.

c) Repeat part (b) assuming that the fraction of the application that is parallelizable is, F , rather than 64%.

2. (25 pts) Recall our on-class discussion of different addressing modes. Among all the nine different addressing modes, which addressing mode(s) is most suitable for the following program variables and operations, and why? (Note that you may list multiple addressing modes for a particular variable/operation type, as long as you can justify the reasons)

a. local variables

b. stack variables

c. array elements

d. pointers

e. branch addresses

f. branch condition evaluation.

3. (25 pts). Using the multi-cycle in order pipeline and its latencies we discussed in slide 1 in 4-Control_Hazard.pdf. Further assume there are two register read ports, one write port, and separate I\$ and D\$ for instruction and data memory accesses. Schedule the following instructions and draw a cycle-wise table similar to slide 15 as we discussed during the class. Mark the data hazard using arrows on the table (also pay attention to structure hazard if any).

fld F4, 0(R2)

fld F5, 8(R2)

fadd.d F6, F4, F5

fadd.d F8, F5, F6

fmult, F7, F6, F4

fsd F7 24(R2)

fsd F8 16(R2)

4. (25 pts) Rename the following code segment (don't forget to also show the overwritten register) assuming the initial map table as shown. The free list starts with p8 and has as many free registers in it as you need to complete the renaming (e.g., p8, p9, p10, ...). On your renamed code indicate (by drawing an arrow from the W to the R) how many true data hazards still exist. After renaming, how many instructions have no true data dependencies (and thus could be executed in parallel with enough functional units)? Note that in this example, you are not asked to maintain the free list.

lw \$1, 40(\$6)

\$1	p1
\$2	p2
\$3	p3
\$4	p4
\$5	p5
\$6	p6

\$7	P7
-----	----

lw \$2, 60(\$6)

lw \$3, 80(\$6)

add \$4, \$2, \$1

mul \$5, \$3, \$4

sw \$5, 80(\$7)

sub \$3, \$4, \$5

sw \$3, 100(\$7)

addi \$6, \$6, 4

addi \$7, \$7, 4